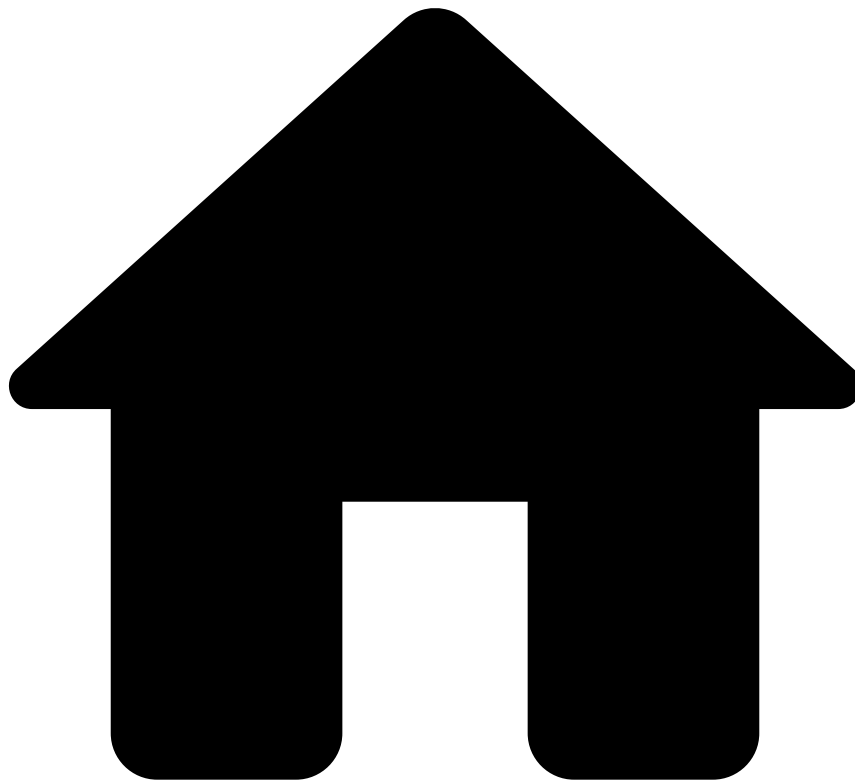


-



- [Core Concepts](#)
- Data Science Loop

On this page

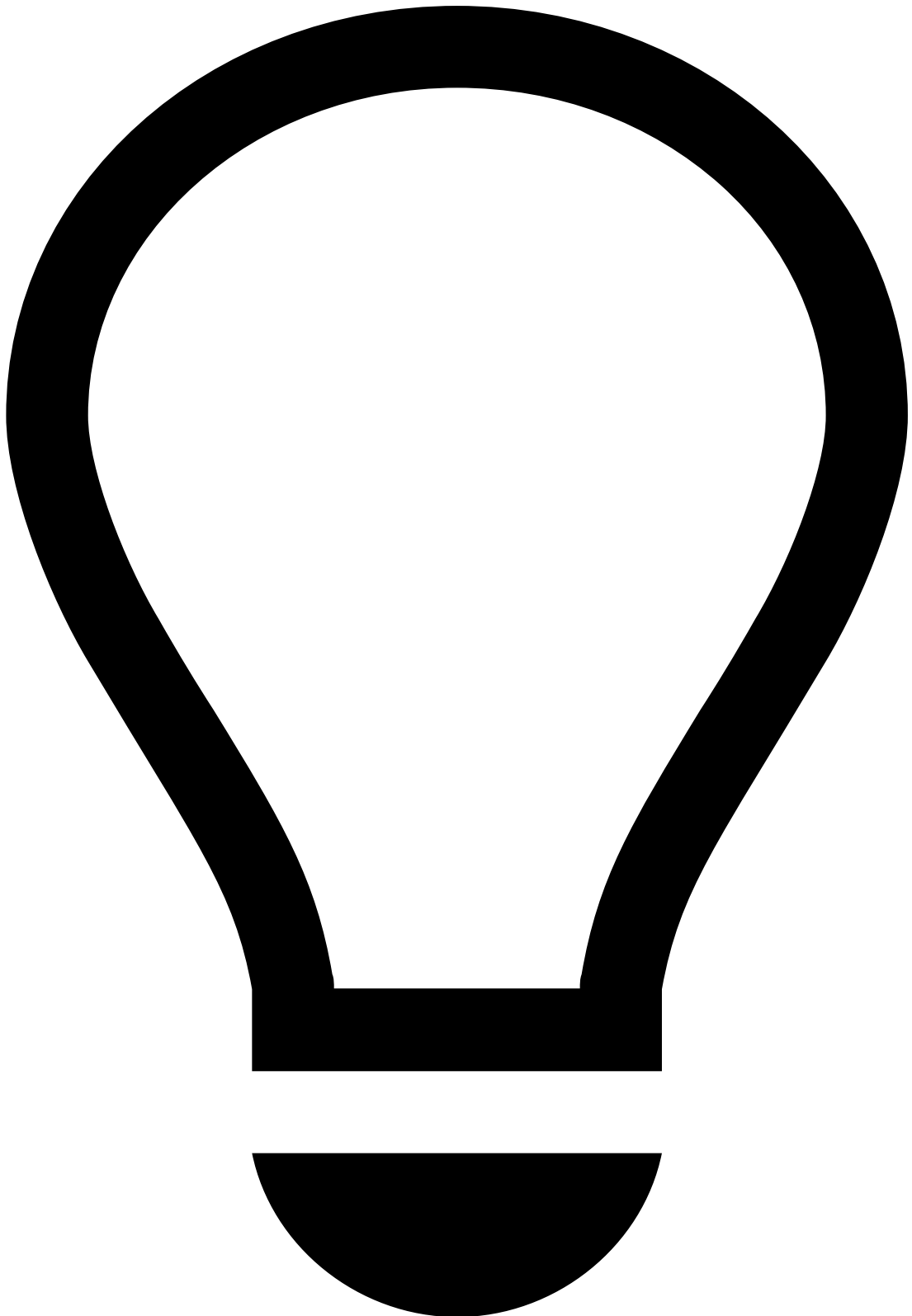
Data Science Loop

Was this page helpful? 

Pulse delivers a full data science platform that is focused on making your data science work much more efficiently than with your traditional toolset. Pulse is designed so that you can easily map your current paradigm to Pulse's concepts, while fully supporting the same data science loop.

Pulse supports the activities in the data science loop by enabling you to:

- Extract deep knowledge over your data to obtain the most value out of it.
- Iterate your work by performing continuous experiments and tuning to extract the best performance out of your model.
- Focus on the real data science by iterating quickly to accelerate your experiments and ultimately obtain better results.



Feedzai IQ

With [Feedzai IQ](#), you can also use ready-to-adopt risk assessment solutions without extensive data science work, thanks to federated financial risk intelligence aggregated from the Feedzai global network.

The following diagram illustrates the sequence of stages in the data science loop.

Data science loop

Data science stages

The data science loop is composed of a sequence of stages with very specific goals and iteration cycles. The following sections explain each of these stages and cycles in more detail.

Additionally, you can learn more about the dedicated environment used in risk assessment strategies according to the data science loop in a dedicated page: [The Data Science Framework \(DSF\)](#).

Clean and load

Clean and load

The main goal of this initial stage is to improve the quality of your raw data so that you can extract meaningful insight from it and ultimately train your model with relevant data. The datasets you receive from your client often have quality issues and a different structure from the one you need to work with.

At this stage you will apply some preprocessing transformations on your data to:

- [Import data](#)

Load your raw historical data, up to Terabyte size, to a system so that you can have access to it and start manipulating it with proper tools. For example, load a CSV file to a database or to a Hadoop cluster.

- [Validate the data schema](#)

Map the raw data to your project's specifications. The raw data can be structured in an outdated or not representative way, so, for example, you may need to change the names of fields or convert the original classification labels into a binary set (e.g. several types of fraud to just *fraud/not fraud*).

- [Clean up the data](#)

Fix inconsistencies in your raw data, remove irrelevant data/fields for data science (usually strings) or join datasets. For example, you may need to join a raw historical transactions dataset with the corresponding labels (e.g. *fraud* or *not fraud*) and set all negative values in transaction amount fields to zero.

This stage is usually performed only once when you receive the historical data from your client.

Analyze and sample

Analyze and sample

During this stage, the goal is to understand your data and prepare it to be intensively transformed and experimented on during the Feature Engineering stage.

At this stage you start to uncover relations in your input data and transform it to be more representative of the problem you are solving to:

- [Identify patterns](#)

Use data visualization tools and apply data visualization techniques over the data to understand it and to discover immediate patterns. For example, correlating the occurrence of fraud with its prevalence in a *Card Not Present* situation.

- [Create dataset sample](#)

Reduce the size of your dataset by generating another one that is better manageable and still representative of the specific problem you are solving. Extremely big datasets are not practical to iterate on because of their size. In addition, there isn't a clear advantage of training machine learning models over those big datasets. In fact, it may even be worse due to the curse of big data, where even noise can be considered a pattern when you have millions of data points. A solution is to balance the dataset by sampling 1%-10% of non-fraud transactions, making it better suited to a machine learning scenario. Reducing its size also reduces the requirements for distributed processing of large volumes of data while making it much easier/quicker to iterate on.

Feature engineering

Feature engineering

Your model decisions are as good as the data that you feed to it during its training. Feature engineering represents the process of transforming the raw data (already cleaned and sampled) into selected and enriched data. The goal is to obtain the dataset that defines what features you will use as input to your machine learning model.

This is the most intelligence-intensive task in the entire cycle and the one where you will probably spend most of your time. Feature engineering is the most important contribution of the data scientist to the model. It includes the following tasks:

- [Select important data for your model](#)

Choose which of the fields in your intermediate dataset are significantly related to the problem you are solving and can be used directly as features. For example, a *card_present* field is much more relevant for a fraud detection scenario than the first name of the customer.

- [Design derived features](#)

Enrich your dataset by highlighting specific characteristics observable in the data. For example, add a boolean *isNational* field based on the transaction country and add enrichment features such as *isDebit*, *isCredit* and *isTorNode* to improve the data with insight from additional external sources.

- [Design metrics \(historical profiles\)](#)

Characterize your data with time-based information that creates a unique profile for each individual data entity. For example, characterize spending habits of customer transactions by merchant type, country, or other field and aggregated over multiple time ranges such as 24 hours, 1 week, 1 month and 6 months.

- [Generate instance/features dataset](#)

Bring all your feature engineering work together by transforming your intermediate data into a single dataset containing all the features you identified/designed. This final dataset is the input for your machine learning model and contains all the relevant data for the model to learn with the appropriate input to solve your target problem, for example, fraud detection.

Build a model

Build a model

The goal of this stage is to take the transformed dataset, including all the computed features, and feed it to a machine learning algorithm in order to produce a model. The model is trained on a subset of the data to learn how to predict future behaviour based on features over historical data and evaluated on a different test dataset.

To create a machine learning model in Pulse, you can:

- [Train a model](#)

Feed your features dataset to teach a machine learning algorithm how to predict future behaviour based on the knowledge you have generated in your dataset. The quality of your model's decisions is a direct consequence of the quality and relevance of the data in the training dataset. Additionally, when you test your model, you must not use any data contained in the training dataset. You should always have a separate dataset, for example, a small subset you extracted from your data before training, as you must always test the model over unknown data.

- [Import a model that you created and trained externally](#)

Use the tools that you are familiar with to build and train your model, and then import it into Pulse.

Optimize model parameters

Within the process of building your model, one recommended step is to tune the model parameters so that you can optimize the performance of your model. There are no guidelines on which parameters to select for a given use case. As such, in order to select an optimum set of parameters, different combinations should be tested to find the one that makes your model operate at its maximum performance.

To adjust your model, you must:

- [Tune the parameters of your model](#)

Iterate your models by configuring different sets of parameter values for each of your algorithm's parameters before training your final model. Try more than one algorithm simultaneously as well to get to the best set of parameters that produces the optimal model for your use case.

Evaluate and compare

Evaluate and compare

At the end of your data science process, you must assess the performance of your trained model. Evaluating a model is not about just generating a single "OK/not OK" metric. It must rather focus on analyzing the results for different metrics, under different perspectives, and for different data subsets. For example, you may want to understand if your model is performing better when catching *Card Present* or *Card Not Present* fraud.

Since this is an iterative process, you can compare the performance of your latest model against the performance of previous iterations and earlier models.

You can also evaluate the impact of each feature on your model with [Feature Importance](#).

To track how close you are to reaching your objectives you can:

- [Evaluate your model](#)

Generate metrics and data visualization objects to let you analyze the performance of your model over an entire test dataset and over different segments and data subsets. Understand the impact of each feature on your model with feature importance. You can use this functionality to identify the subset of variables that contribute the most and the least to the model predictions. As a result, you can reduce model complexity and computation time. Understanding which features have higher impact on the model might also help you to identify new relevant features.

- [Compare your models](#)

Understand how the current iteration of your model performs against previous versions and how close your current performance is to reaching your project's goals. You can compare metrics such as Precision, Recall and False Positive Rate (FPR), cost-based metrics such as fraud money Recall, ROC curves, etc. You can also compare the results of feature importance between two model evaluations.

In fact, while you are trying to reach your business objectives, evaluating, comparing and applying feature importance to your models does not end the data science process. It just closes the data science loop and starts a new iteration. Usually, lots of iterations are required to reach your final goal. You will often uncover new patterns, gain new insight, re-engineer your features and test different parameter combinations to maximize the performance of your model.

Iteration occurs at the following stages:

Model building and parameter optimization

Model building and parameter optimization

The most common iteration is to tune the parameters of your model with different parameter configurations and retrain it on the same dataset. A model has lots of parameters to configure and much more combinations that you can try out before you feel that the model is delivering top performance.

The approach that Feedzai uses to optimize parameters is to perform a grid search (parameter sweep) where Pulse creates a model for every combination of values.

Feature engineering

Feature engineering

You can reiterate your data science work from the Feature engineering stage. You can try to improve your model by extracting extra/different features from the original data that better correlate with the problem you are solving, for example, fraud detection, define more profiles and further enriching the data.

Getting average results is usually fairly straightforward when you already have some domain knowledge and compute a set of generic features. However, pushing a model to its optimal performance, and addressing the long tail of the problem is very hard and requires many Feature engineering iterations. You'll most likely end up designing much more features, that are very specific to your use case and data than you did for the initial iterations.

Data analysis and sampling

Data analysis and sampling

Reiterating your (almost) entire project is more common in the earlier stages when you are still unfamiliar with the available data. As you know the dataset better, your need to re-analyze the data will gradually decrease.

It is also common that you only uncover problems in your sampled data after you complete a full iteration and evaluate the results against external data, causing you to have to resample it and better balance it for your specific use case.

Automate data science tasks

To accelerate the data science loop, you can automate some of the above tasks.

In Pulse, there are two possibilities for that:

1. Automate steps 2-5 with AutoML

The [AutoML](#) feature of Pulse automates almost the whole data science loop, from step 2 (Analyze & Sample) to step 5 (Evaluate & Compare). It helps at the kickoff of a new data science project, or when you want to augment the feature set of an existing model. [Start an AutoML run](#) by defining the starting data source, the metric to optimize, and some other parameters. AutoML does the following for you:

- Creates a feature [Plan](#) and executes it, resulting in a new dataset with the engineered features.
- Creates a Hyperparameter Tuning to train models, and to evaluate them. AutoML prepares the best model for usage.

2. Automate steps 4-5 with Hyperparameter Tuning

Use [Hyperparameter Tuning](#) to automate steps 4-5 of the data science loop: model building, parameter tuning, and model evaluation. To [start a Hyperparameter Tuning job](#), select a set of ML algorithms and the parameter values you want to test them with. As a result, Pulse does the following for you in an automatized way:

- Trains all the selected models with all the defined parameter values (parameter sweep).
- Evaluates the trained models.

Runtime activities

Runtime activities

When your data science work is finished and the data science loop is closed, it's time to deploy it to Production. These deployment activities are usually reserved for engineers as the runtime environment is greatly different from the offline data science stage.

Integrating the results of the data science loop with the runtime environment is usually very challenging, time-consuming and requires advanced technical skills. It's also an iterative process of deploying, evaluating and benchmarking the results against the expected performance as designed during the data science stage. For every model that needs to be deployed, new engineering work is required.

It may take up to several months for engineers to be able to ensure that your data science work is correctly translated to runtime, maintaining the same semantics for data transformation, time-based profiles, enrichment and any feature, and that it yields the same results.

However, Pulse already integrates the data science and runtime environments making it much easier and quicker to deploy a project to Production. This makes runtime iterations much faster and safer as Pulse ensures that data science semantics are correctly translated to Production without requiring you to have a set of very sophisticated technical skills.

Learn more about [how Pulse projects work](#) and how they integrate the data science loop and runtime tasks.

